

MACHINE LEARNING IN FINANCE

MODULE 3 | LESSON 4

TREES IN PRACTICE

Reading Time 12 minutes

Prior Knowledge Tree-based machine learning methodology

Keywords Model fitting, Training models, Testing models, Model performance, ROC, Area Under the Curve

In the previous lesson, we looked at how tree-based algorithms can be used to predict class outcomes. We now apply this theory to predict one-month returns of a group of stocks exceeding the median or not.

1. Trees in Practice

Recall from Module 3, Lesson 3 that given a set of data with predictors and a target variable with class outcomes, we can fit a decision tree classifier that can intuitively show the rules or criteria that makes a target class highly likely. This non-black box characteristic is a benefit of decision trees as we will show in this lesson by fitting a tree to stock data in order to predict whether a future one-month return will change by more than a threshold or not, i.e., we consider the same binary class problem in lesson 2 of this module.

We therefore use the same data (from Coqueret and Guida chapter 9, section 9.4), specifically the future one-month returns data for a collection of stocks and financial data.

2. The Data

The data can be accessed via

<https://github.com/shokru/mlfactor.github.io/tree/master/material>. Recall that the features we look into are:

- *Mkt_Cap_12M_Usd* - Size
- *Pb* - Price to book
- *Vol1Y_Usd* - For share turnover
- *Return_On_Capital*

- *Roe* - Return on equity
- *Pe* - Price to Earnings ratio

We use Python to perform the analysis and take the same data from the year 2000 to 2018 with only 50 stock IDs. Let us read in the data as in Lesson 2 and take a preview.

```
In [1]: import numpy as np
import pandas as pd

# Location of the data

# ENTER LOCATION OF DATA FILE BELOW
df_50 = pd.read_csv("mlfac_dat.csv")

# consider data from 2000 to 2018

df_50 = df_50[(df_50["date"] > "1999-12-31") & (df_50["date"] < "2019-01-01")]

# Filter on the first 50 stock ids

df_50 = df_50[df_50["stock_id"] <= 50]

# Created a class variable for 1-Month returns if
# changes by more than a threshold = thresh

thresh = 0.04
df_50["R1M_Usd_C"] = np.where(df_50["R1M_Usd"].abs() > thresh, 1, 0)

# Make the response variable into integer format

df_50["R1M_Usd_C"] = df_50["R1M_Usd_C"].astype(int)

# Features of interest
feature_cols = ["Mkt_Cap_12M_Usd", "Pb", "Vol1Y_Usd", "Return_On_Capital", "Roe"

df_50 = df_50[feature_cols + ["R1M_Usd_C"]]

df_50.head()
```

```
Out[1]:
```

	Mkt_Cap_12M_Usd	Pb	Vol1Y_Usd	Return_On_Capital	Roe	Pe	R1M_Usd_C
0	0.52	0.21	0.77	0.08	0.02	0.5	0
1	0.52	0.21	0.73	0.08	0.02	0.5	1
2	0.51	0.15	0.67	0.10	0.06	0.5	0
3	0.51	0.15	0.63	0.10	0.06	0.5	1
4	0.52	0.15	0.54	0.10	0.06	0.5	1

We see 6 predictors with the target variable being the one-month future returns (R1M_Used_C), that is a binary variable taking on 1 if the return is more than 4% in either a long or short direction and 0 otherwise. Keep in mind that features have been pre-selected in this example. In practice, however, feature selection generally takes a significant time to perform and often adds much value in improving performance.

3. Train the Tree

Now that we have a view of the data, we will proceed with splitting the dataset into a train/test 80/20 split.

```
In [2]: from sklearn.model_selection import train_test_split

# split dataset in features and target variable
X = df_50[feature_cols] # Transformed Features

y = df_50["R1M_Used_C"] # Target variable

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

We now declare our decision tree classifier using the `sklearn` package.

```
In [3]: # Import Decision Tree Classifier

from sklearn.tree import DecisionTreeClassifier

# Create Decision Tree classifier object

clf = DecisionTreeClassifier(criterion="gini", max_depth=3, min_samples_leaf=25)

# Train Decision Tree classifier

clf = clf.fit(X_train, y_train)
```

The decision tree produces a set of rules that are easily interpretable in our case since we have used only a few features for illustrative purposes. A flow diagram of the decision tree can be shown using the code below, which will produce a flow diagram as in Figure 1. This shows that companies with turnover above 0.905 are 78% likely to have returns that change by more than 4%. This is a strong differentiating node.

```
In [4]: # visualize the tree

import matplotlib.pyplot as plt
from sklearn import tree

# features we used

fn = feature_cols

# labels of the target class

cn = ["0", "1"]
```

```
fig = plt.figure(figsize=(25, 20))
_ = tree.plot_tree(clf, feature_names=fn, class_names=cn, filled=True)

# save the figure to file

# fig.savefig('imagename.png')
fig.savefig("decistion_tree.png")
```

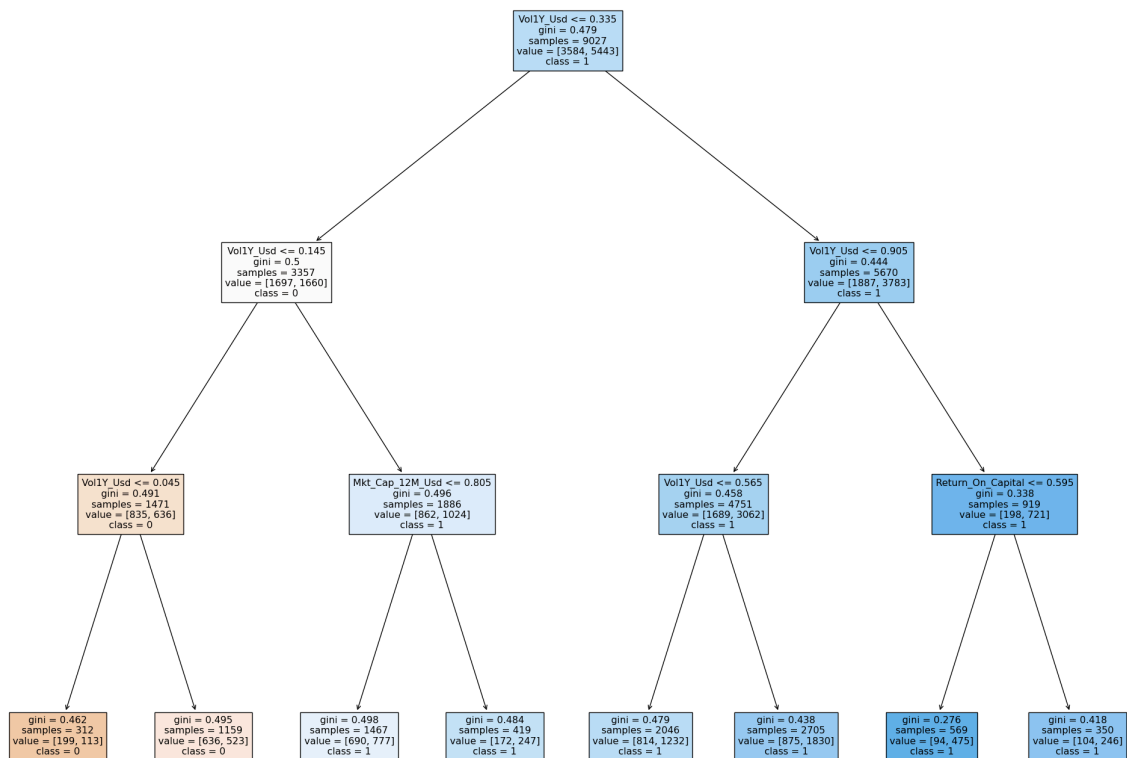


Fig. 1: Flow Diagram of Decision Tree for Predicting Price Change of More than 4% in Any Direction

Figure 1 shows an absolute change exceeding 4% as class 1 and class 0 otherwise. The blue nodes represent cases that majority have class 1, i.e., the darker the shade of blue, the purer the node. Similarly, for Class 0, the more orange the node, the higher proportion of Class 0.

It is worth pointing out that the cuts in Trees are parallel to existing axis whereas in SVM the cuts can be, but not restricted to, a linear combination of features. The sequential nature of the splitting shown in Fig 1 shows that only one feature can be used at a time. SVM on the other hand is not restricted by this.

4. Tree Performance

We've trained our decision tree classifier and we are now ready to test the performance. To do this, we need to run the unseen test dataset through the classifier to determine the

class predictions for the target variable. It would be insightful to compare this to a random guess classifier, i.e., a model that requires no skill.

In [5]: *# Predict the response for test dataset*

```
y_pred = clf.predict(X_test)

# Obtain the probabilities
probs_tmp = clf.predict_proba(X_test)

probs = probs_tmp[:, 1]
```

Fortunately, `Sklearn` has tools to evaluate performance metrics for most machine learning algorithms. We look at the ROC curve and AUC as in Lesson 2.

In [6]: **import** matplotlib.pyplot **as** plt
from sklearn.metrics **import** roc_auc_score, roc_curve

```
# plot the roc curve for the model

# generate a no skill or random guess prediction

ns_probs = [0 for _ in range(len(y_test))]

# calculate roc curves for random guess

ns_fpr, ns_tpr, _ = roc_curve(y_test, ns_probs)

# calculate roc curves for the decision tree

fpr, tpr, _ = roc_curve(y_test, probs)

# calculate scores

ns_auc = roc_auc_score(y_test, ns_probs) # random guess

tree_auc = roc_auc_score(y_test, probs) # tree classifier

# summarize scores

print("No Skill: ROC AUC=%.3f" % (ns_auc))

print("Classifier: ROC AUC=%.3f" % (tree_auc))

plt.plot(ns_fpr, ns_tpr, linestyle="--", label="No Skill")

plt.plot(fpr, tpr, marker=".", label="Tree Classifier")

# axis labels

plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

# show the legend

plt.legend()
```

```
# show the plot
```

```
plt.show()
```

No Skill: ROC AUC=0.500

Classifier: ROC AUC=0.606

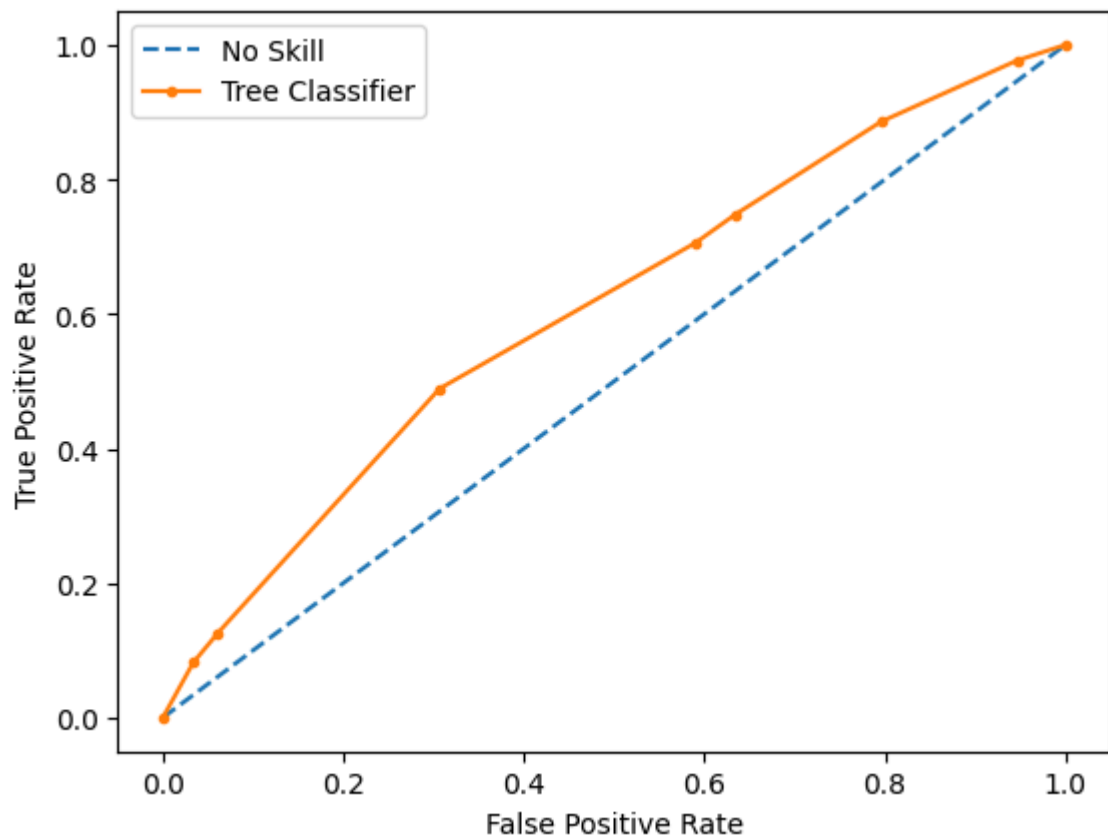


Fig. 2: ROC Curve of the Decision Tree Classifier vs. a Random Guess or No Skill Predictor

The AUC for the classifier is 60.6%, which is better than a random guess. It is worth noting that a subset of predictors was chosen to illustrate the decision tree flow diagram in Figure 1. Feature selection plays an important part in model performance and should not be overlooked.

Lastly, it is worth looking at the implementation of a tree-based algorithm that uses boosting. The algorithm we consider is popular in the Kaggle community and can be easily implemented in Python using the `xgboost` package. The hyperparameters have been specified, but not optimized, instead of the default settings. For further reading on these parameters, please refer to ("[XGBoost Parameters](#) — [Xgboost 1.6.1 Documentation](#)")

```
In [7]: import xgboost as xgb

# Init classifier
xgb_cl = xgb.XGBClassifier(
    max_depth=2,
    eta=0.1, # Learning rate
    objective="binary:logistic",
    eval_metric="auc",
)
```

```

# Fit
xgb_cl.fit(X_train, y_train)

# Predict
preds_xgb = xgb_cl.predict(X_test)

# Obtain the probabilities
preds_prob_xgb_tmp = xgb_cl.predict_proba(X_test)

preds_prob_xgb = preds_prob_xgb_tmp[:, 1]

```

The results of the `xgboost` model along with the decision tree and random guess classifier is shown in figure 3 below. The `XGBoost` ties with the decision tree at 60.6%. The reader, as an exercise, can decrease the max depth of the tree to determine the change in performance. This will show the sensitivity to the hyperparameters.

```

In [8]: # generate a no skill or random guess prediction
ns_probs = [0 for _ in range(len(y_test))]
# calculate roc curves
# random
ns_fpr, ns_tpr, _ = roc_curve(y_test, ns_probs)
# tree
fpr, tpr, _ = roc_curve(y_test, probs)
# xgb
xgb_fpr, xgb_tpr, _ = roc_curve(y_test, preds_prob_xgb)

# calculate scores
ns_auc = roc_auc_score(y_test, ns_probs) # random guess
tree_auc = roc_auc_score(y_test, probs) # tree classifier
xgb_auc = roc_auc_score(y_test, preds_prob_xgb) # tree classifier
# summarize scores
print("No Skill: ROC AUC=%.3f" % (ns_auc))
print("Tree Clasifier: ROC AUC=%.3f" % (tree_auc))
print("XGB Clasifier: ROC AUC=%.3f" % (xgb_auc))

plt.plot(ns_fpr, ns_tpr, linestyle="--", label="No Skill")
plt.plot(fpr, tpr, marker=".", label="Tree Classifier")
plt.plot(xgb_fpr, xgb_tpr, marker=".", label="XGB Classifier")
# axis labels
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
# show the legend
plt.legend()
# show the plot
plt.show()

```

```

No Skill: ROC AUC=0.500
Tree Clasifier: ROC AUC=0.606
XGB Clasifier: ROC AUC=0.606

```

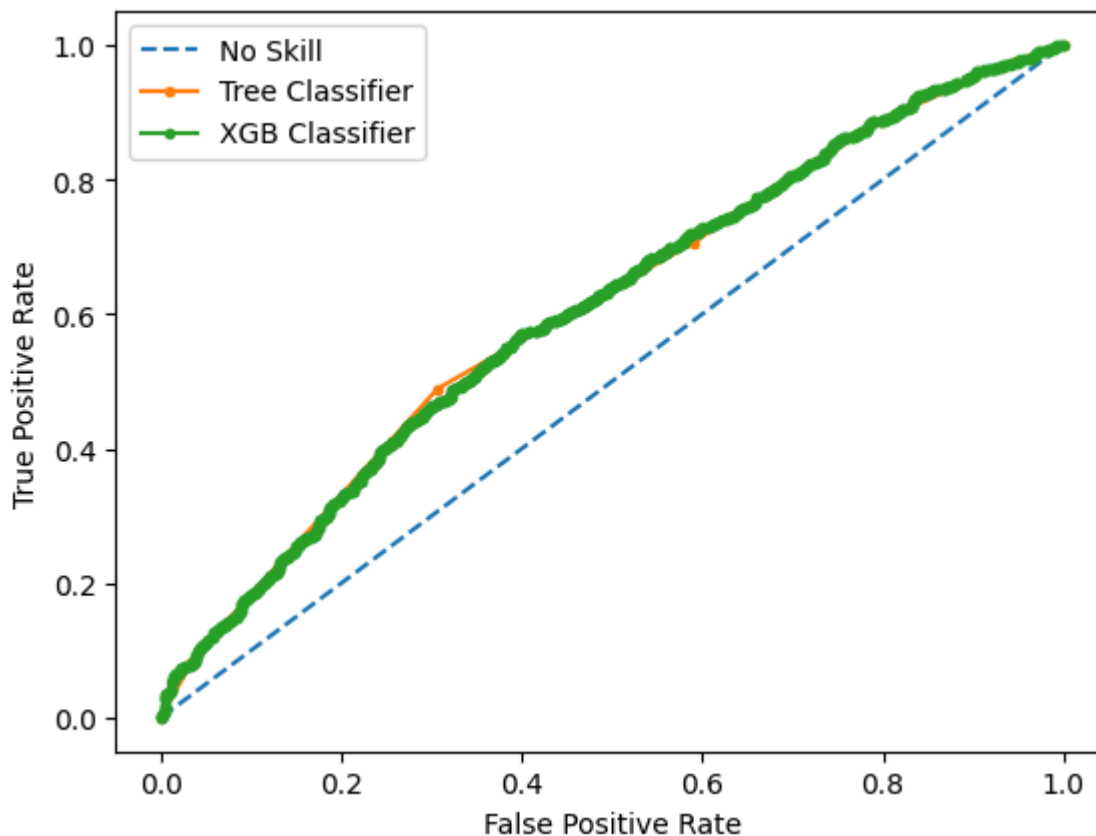


Fig. 3: ROC Curves for the Tree, XGBoost, and No-Skill Model

5. Conclusion

The examples illustrated in this lesson help us understand the tree-based algorithms from the simple decision tree to the `xgboost` model. The model performance is better than a random guess thus showing added benefit. This lesson focused on showing how to apply tree-based algorithms to a real-world classification problem. A model that does better than a random guess can be easily motivated; however, keep in mind the goal of the specific classification problem, as it may require a minimum AUC to be achieved. The next module will explore support vector machines and neural networks, another class of predictive models.

References

- Coqueret, Guillaume, and Tony Guida. *Machine Learning for Factor Investing: R Version*. CRC Financial Mathematics Series, Chapman and Hall/CRC, 2022.
- "XGBoost Parameters — Xgboost 1.6.1 Documentation." XGBoost Parameters, <https://xgboost.readthedocs.io/en/stable/parameter.html>. Accessed 30 June 2022.